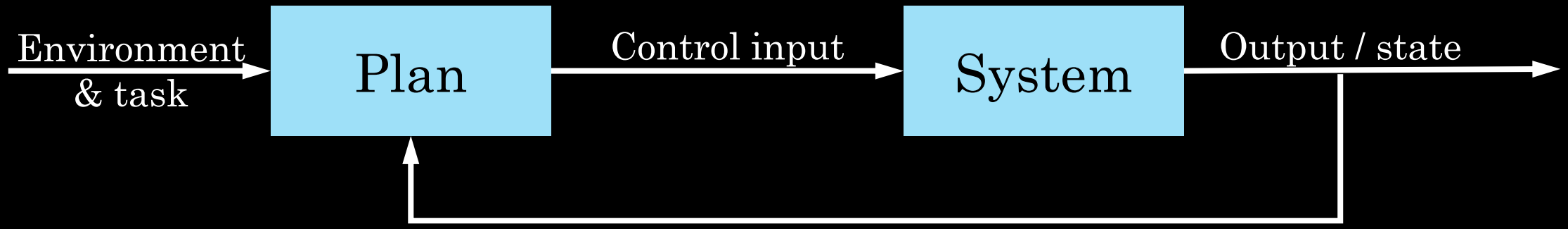
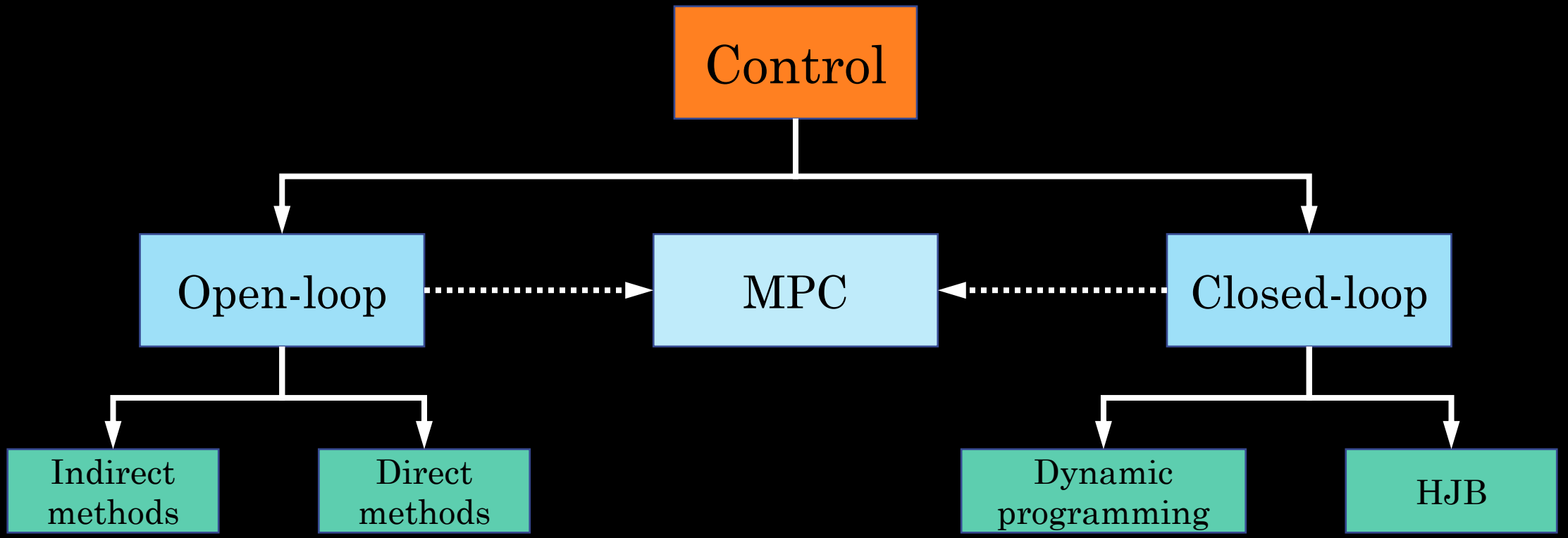
Closed-loop vs open-loop control

Open-loop control: No feedback loop

Closed-loop control: Yes feedback loop




Control
Open-loop
MPC
Closed-loop
Indirect methods
Direct methods
Dynamic programming
HJB

Sequential decision making

Dynamic programming // Closed-loop control

Additional reference: https://underactuated.mit.edu/dp.html

How should we make optimal decisions which have consequences on the future?

State = [academic, professionalism, well-being, finances]

First day of school	End of Freshman	End of Sophomore	End of Junior	End of Senior
Study every day	Study every day	Study every day	Study every day	Apply for job
Join research lab	Join research lab	Join research lab	Join research lab	Apply to grad school
Attend parties	Attend parties	Attend parties	Attend parties	Travel the world
Work two jobs	Work two jobs	Work two jobs	Work two jobs	Create start-up
Overcommit to RSO	Overcommit to RSO	Overcommit to RSO	Overcommit to RSO	Attend parties

How should we make optimal decisions which have consequences on the future?

State = [academic, professionalism, well-being, finances]

Study every day

Join research lab

Attend parties

Work two jobs

Overcommit to RSO

Apply for job

Apply to grad school

Travel the world

Create start-up

Attend parties

End of Senior

Retirement

Sequential decision making

- **Actions have consequences**. The consequences may be immediate or delayed.
  - E.g., Attending party before exam
    - Immediate: Lots of fun, improve mood and happiness
    - Delayed: Not alert during exam, perform poorly, not get a good grade, miss out on Dean's list.
- Hard to make optimal decisions
  - Uncertainty in future (e.g., exam may be easy)
  - Large state space (many possible situations could arise)
  - Hard to measure exactly the consequences

Group activity

- Describe some “real-world” settings where you need to perform sequential decision making
  - Immediate consequence
  - Future consequence
  - States and decisions/controls
  - Potential sources of uncertainties, and potential ways to address them
- Example: Chess
  - Taking opponent’s pieces
  - Setting up board to be in a position to win
  - Location of pieces
  - Piece to move
  - What the opponent does on their move
    - Could assume opponent is optimal

Are there SDM problems with nice structure that make it easier to solve?

Principle of Optimality

- An optimal solution contains within its optimal solutions to its subproblems
- Given an optimal solution to a SDM problem, then any tail portion of the solution is also optimal to that sub SDM problem
- When following an optimal solution, the remainder of the solution must also be optimal from that state onwards.
- If the problem is solved optimally, then regardless of past decisions, future decisions must be optimal for the current state.

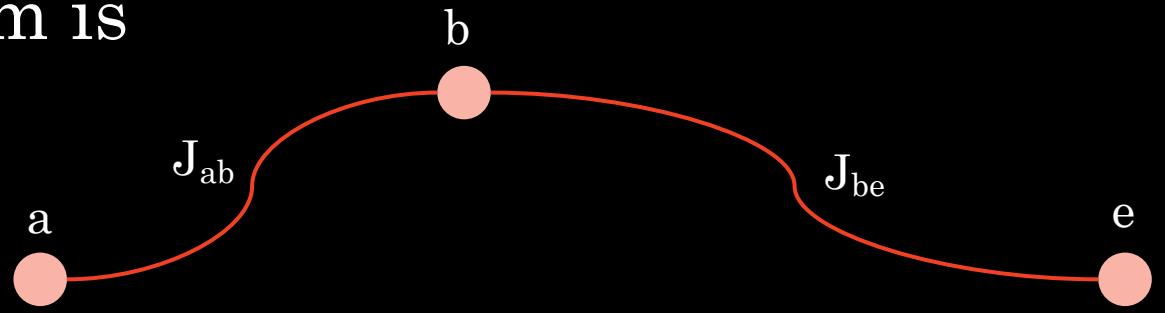
How should we make optimal decisions which have consequences on the future?

State = [academic, professionalism, well-being, finances]

First day of school	End of Freshman	End of Sophomore	End of Junior	End of Senior
Study every day	Study every day	Study every day	Study every day	Apply for job
Join research lab	Join research lab	Join research lab	Join research lab	Apply to grad school
Attend parties	Attend parties	Attend parties	Attend parties	Travel the world
Work two jobs	Work two jobs	Work two jobs	Work two jobs	Create start-up
Overcommit to RSO	Overcommit to RSO	Overcommit to RSO	Overcommit to RSO	Attend parties

Principle of optimality: Setup

- Suppose optimal path for a multi-stage decision-making problem is

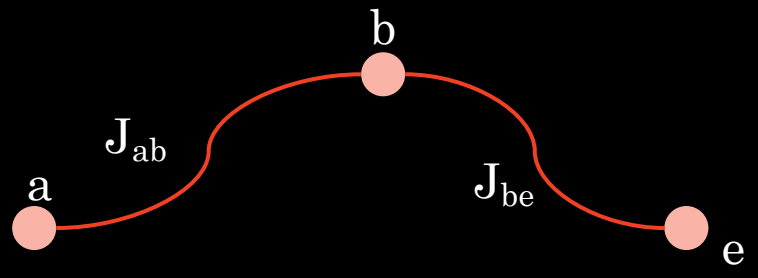


- First decision yields segment $a - b$ with cost J_{ab}
- Remaining decisions yield segment $b - e$ with cost J_{be}
- Optimal cost is then $J_{ae}^{*} = J_{ab} + J_{be}$
- **Assumption**: Cost has *additive* structure

Adapted from Stanford AA 203 course notes

Principle of optimality

- **Claim**: If $a - b - e$ is the optimal path from a to e, then $b - e$ an optimal path from b to e
- **Proof**: By contradiction



Implications of the Principle of Optimality

- To find optimal decision sequence, we consider solving tail subproblems.
- Then consider solving the tail subproblems of those subproblems, and so forth.
- Start with the smallest tail subproblem and work our way backwards
- Can ignore the past, just concerned with optimizing the future from the current situation
  - Need to solve for *all* tail subproblems

Dynamic programming

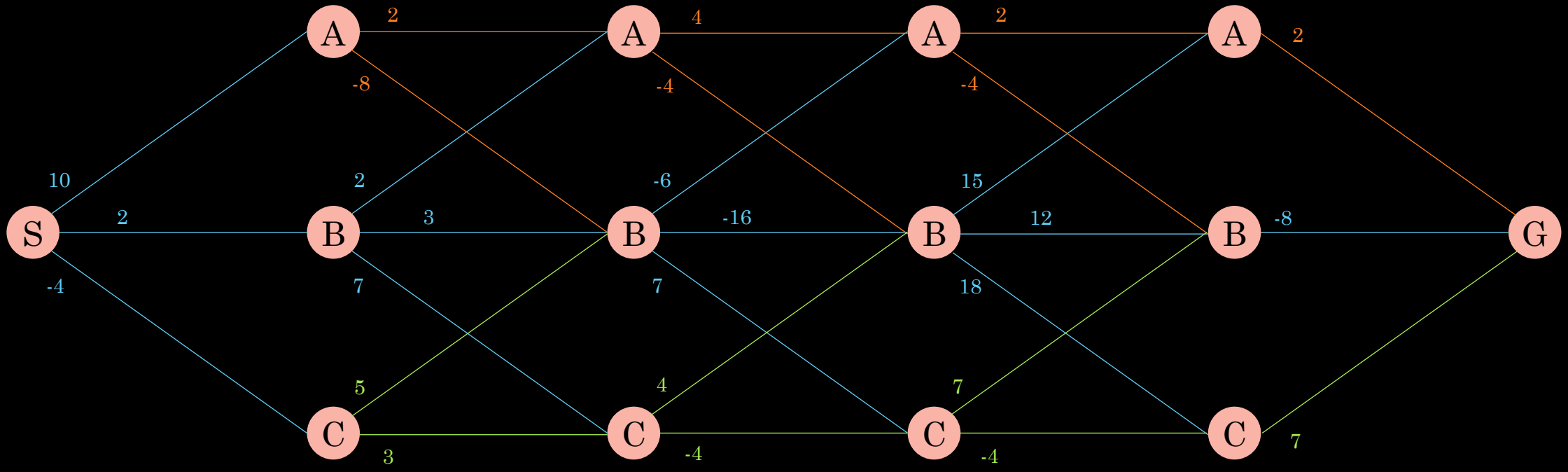
- Algorithmic technique for solving complex problems by breaking them down into simpler subproblems
- **Core idea**: Solve each subproblem only once and store the solution
- **Benefit**: Prevents redundant computations when subproblem is encountered again

- A common technique to solve many problems, not just control problems

Example of dynamic programming in other fields

- 💻 Dividing text of paragraph into lines of approximately equal length—Knuth–Plass line-breaking algorithm used dynamic programming
- 🚚 Route optimization, traveling salesman problem, orienteering problem. Useful for delivery route planning, surveillance missions, transportation routing, etc
- 🧬 Sequence alignment: Comparing DNA or protein sequences to find similarities (e.g., Needleman–Wunsch or Smith–Waterman algorithms).

Example



Key idea: Keep track of a *value function*

Value function

A "score" that measures the long-term potential of that state, assuming a fixed policy π is used to from that point forward.

- "Cost-to-go": The future accumulated cost (or reward) from following policy π
- Combines immediate and future consequences

Trajectory optimization as a SDM

Consider the discrete time setting (easier to start with)

- **System:** $\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k), k = 0, \ldots, N-1$
- **Control constraints:** $\boldsymbol{u}_k \in U(\boldsymbol{x}_k)$.
- **Cost:** $J(\boldsymbol{x}_0, \boldsymbol{u}_{0:N-1}, t_0) = g_N(\boldsymbol{x}_N) + \sum_{k=0}^{N-1} g(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k)$
- Let $\{\boldsymbol{u}_0^\star, \boldsymbol{u}_1^\star, \ldots, \boldsymbol{u}_{N-1}^\star\}$ be an *optimal* control sequence starting at state $\boldsymbol{x}_0$, with corresponding state sequence $\{\boldsymbol{x}_0^\star, \boldsymbol{x}_1^\star, \ldots, \boldsymbol{x}_N^\star\}$

Take a closer look at the cost

Let $\boldsymbol{u}_k^\star = \pi^\star(\boldsymbol{x}_k, t_k)$ be the optimal policy

Cost at t_0 $(k = 0)$

$$J^\star(\boldsymbol{x}_0, t_0) = g_N(\boldsymbol{x}_N) + \sum_{k=0}^{N-1} g(\boldsymbol{x}_k, \pi^\star(\boldsymbol{x}_k, t_k), t_k)$$

By Principle of Optimality

$$J^\star(\boldsymbol{x}_j, t_j) = g_N(\boldsymbol{x}_N) + \sum_{k=j}^{N-1} g(\boldsymbol{x}_k, \pi^\star(\boldsymbol{x}_k, t_k), t_k)$$

$$J^\star(\boldsymbol{x}_j, t_j) = g\big(\boldsymbol{x}_j, \pi^\star(\boldsymbol{x}_j, t_j), t_j\big) + g_N(\boldsymbol{x}_N) + \sum_{k=j+1}^{N-1} g(\boldsymbol{x}_k, \pi^\star(\boldsymbol{x}_k, t_k), t_k)$$

$$J^\star(\boldsymbol{x}_j, t_j) = g\big(\boldsymbol{x}_j, \pi^\star(\boldsymbol{x}_j, t_j), t_j\big) + J^\star(\boldsymbol{x}_{j+1}, t_{j+1})$$

Value function represents *cost-to-go*

Cost-to-go, also referred to as the **Value Function**

- The future total cost at $(\boldsymbol{x}_k, t_k)$ if following policy π

$$V_\pi(\boldsymbol{x}_k, t_k) = g_N(\boldsymbol{x}_N) + \sum_{k=k}^{N-1} g(\boldsymbol{x}_k, \pi(\boldsymbol{x}_k, t_k), k) \coloneqq J(\boldsymbol{x}_j, t_j; \pi)$$

$$J(\boldsymbol{x}_j, t_j; \pi) = g(\boldsymbol{x}_j, \pi(\boldsymbol{x}_j, t_j), t_j) + J(\boldsymbol{x}_{j+1}, t_{j+1}; \pi)$$

$$V_\pi(\boldsymbol{x}_k, t_k) = g(\boldsymbol{x}_k, \pi(\boldsymbol{x}_k, t_k), t_k) + V_\pi(\boldsymbol{x}_{k+1}, t_{k+1})$$

How to compute the *optimal* policy?

Previously, it was assumed a policy was provided.

We can simultaneously compute the optimal policy while computing the value function.

$$V_\pi(\boldsymbol{x}_k, t_k) = g(\boldsymbol{x}_k, \pi(\boldsymbol{x}_k, t_k), t_k) + V_\pi(\boldsymbol{x}_{k+1}, t_{k+1})$$

$$V^\star(\boldsymbol{x}_k, t_k) = \min_{\boldsymbol{u}_k \in \mathcal{U}} g(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k) + V^\star(f(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k), t_{k+1})$$

Okay, but how exactly do we compute the Value Function?

Assumes we have V at the next time step

$$V_\pi(\boldsymbol{x}_k, t_k) = g(\boldsymbol{x}_k, \pi(\boldsymbol{x}_k, t_k), t_k) + V_\pi(\boldsymbol{x}_{k+1}, t_{k+1})$$

$$V^\star(\boldsymbol{x}_k, t_k) = \min_{\boldsymbol{u}_k \in \mathcal{U}} g(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k) + V^\star(f(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k), t_{k+1})$$

Need a *terminal* case (aka base case)

When $k = N$?

$$V^\star(\boldsymbol{x}_N, t_N) =$$

Base case: Use terminal cost

$$V^\star(\boldsymbol{x}_N, t_N) = g_N(x_N)$$

$$V^\star(\boldsymbol{x}_{N-1}, t_{N-1}) = \min_{\boldsymbol{u}_{N-1} \in \mathcal{U}} g(\boldsymbol{x}_{N-1}, \boldsymbol{u}_{N-1}, t_{N-1}) + V^\star(f(\boldsymbol{x}_{N-1}, \boldsymbol{u}_{N-1}, t_{N-1}), t_N)$$

$$V^\star(\boldsymbol{x}_{N-2}, t_{N-2}) = \min_{\boldsymbol{u}_{N-2} \in \mathcal{U}} g(\boldsymbol{x}_{N-2}, \boldsymbol{u}_{N-2}, t_{N-1}) + V^\star(f(\boldsymbol{x}_{N-2}, \boldsymbol{u}_{N-2}, t_{N-2}), t_{N-1})$$

Dynamic programming: Summary

- Work *backward* in time
- Start with terminal cost: $V^\star(\boldsymbol{x}_N, t_N) = g_N(x_N)$
- Loop backward in time
  - $V^\star(\boldsymbol{x}_k, t_k) = \min_{\boldsymbol{u}_k \in \mathcal{U}} g(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k) + V^\star(f(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k), t_{k+1})$
  - $\pi^\star(\boldsymbol{x}_k, t_k) = \operatorname{argmin}_{\boldsymbol{u}_k \in \mathcal{U}} g(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k) + V^\star(f(\boldsymbol{x}_k, \boldsymbol{u}_k, t_k), t_{k+1})$
- Final outputs
  - $V^\star(\boldsymbol{x}_k, t_k)$ for all at states and all time
  - $\pi^\star(\boldsymbol{x}_k, t_k)$ for all at states and all time